Mobile Solutions

Student Name: Vsevolod Svinin

App Title: WorkSpot

Tasks:

Description of App incl. Sketches 2

Create a digital Mock-Up 6

Configure your Development Environment 8

Develop the Requirements and Steps for App Distribution 9

Implement your Design Proposal 10

Print as a PDF Document 17

Description of App incl. Sketches

Create a **Design Proposal** for an Application with the following Requirements:

- Develop and **describe** the **Idea for an Application** (at least half a page of text).
- Define the **User Target Group(s).**
- Design at least **6 Screens on Paper** (Sketches) and **insert images** of your sketches here.

Description of WorkSpot:

WorkSpot is a mobile application designed to help students, freelancers, remote workers and other people find suitable places for studying and working in their city. The application provides a map-based overview of cafés, coworking spaces, libraries and other locations where users can work with a laptop or study.

Unlike conventional map applications, WorkSpot focuses specifically on the factors that are important when choosing a place to work or study. Users can see an overall rating of a location as well as separate ratings for noise level, seating and table comfort, Wi-Fi quality and availability of power outlets. This allows users to choose a location according to their individual needs instead of relying only on a general rating.

Each location also contains additional information such as price, opening hours and type of location. The price is displayed separately from the user rating. A location can be marked as “Free” if it can be used without payment or a required purchase, while paid locations can be indicated using the € price-level system. Opening hours show users when the location is available, for example, “08:00–22:00”, together with an “Open now” or “Closed” status.

WorkSpot is designed as a platform for both users and businesses. Users can search for locations, view their information, leave reviews and rate locations according to the different criteria. Cafés, coworking spaces, libraries and other organizations can also add their own locations to the platform and provide basic information such as address, opening hours and price level. However, the ratings and reviews are provided by users in order to reflect their actual experience.

The main purpose of WorkSpot is to make it easier to answer a simple question: “Where can I comfortably work or study right now?” The application combines a city map with practical information and user-generated ratings specifically focused on the working and studying experience.

Target Users

1. Students - looking for quiet and affordable places to study.
2. Freelancers - looking for comfortable places to work outside their home.
3. Remote workers - looking for cafés and coworking spaces with good Wi-Fi and power outlets.
4. Travelers/digital nomads - looking for suitable places to work while visiting a city.

Business / Organization Users

- Café owners
- Coworking space operators
- Libraries
- Other organizations providing work/study spaces

Screen 1 - Map

The main screen displays a city map with different locations where users can work or study.

Users can see:

- Cafés
- Coworking spaces
- Libraries
- Other suitable locations

Each location is represented by a marker on the map. Users can tap a marker to view more information about the selected place.

Screen 2 - Place Details

This screen displays detailed information about a selected location.

It includes:

- Overall user rating
- Noise level
- Table and seating comfort
- Wi-Fi quality
- Availability of power outlets
- Price level
- Opening hours
- Current status: Open / Closed

Users can also access the reviews or rate the location themselves.

Screen 3 - Reviews

This screen displays reviews and ratings from other users.

Users can see:

- Written reviews
- Individual ratings for different criteria
- Overall rating
- Date of the review

This helps users understand the actual experience of working or studying at the location.

Screen 4 - Rate a Place

Users can rate a location based on several criteria:

- Noise level
- Table & seating comfort
- Wi-Fi quality
- Power outlet availability

Users can also write a short review and submit their rating.

Screen 5 - Add a Place

This screen allows businesses, organizations, or users to add a new location to the application.

The form can include:

- Name of the place
- Address
- Type: Café / Coworking Space / Library
- Price: Free / € / €€ / €€€
- Opening hours
- Basic information about the location

The submitted location can then be added to the map.

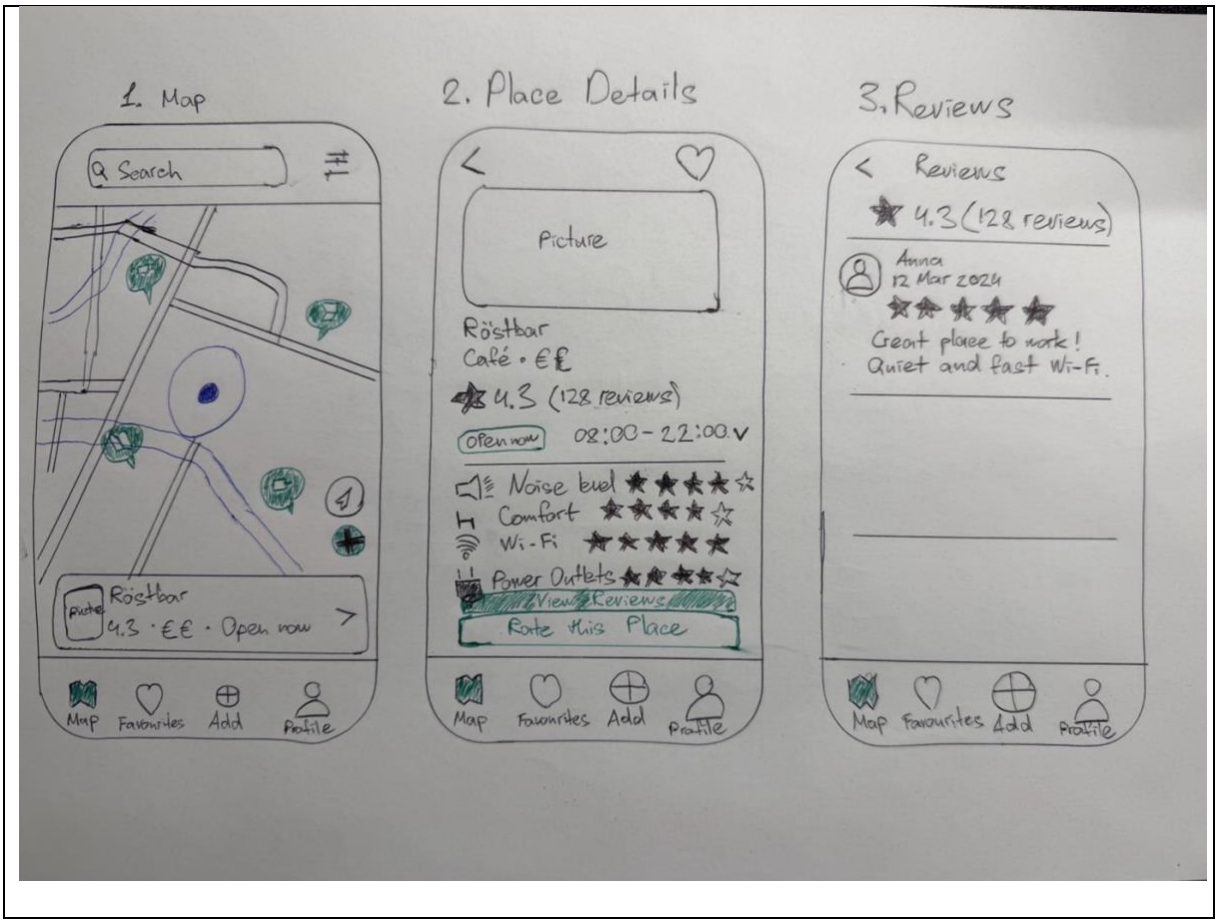
Screen 6 - Profile / Favorites

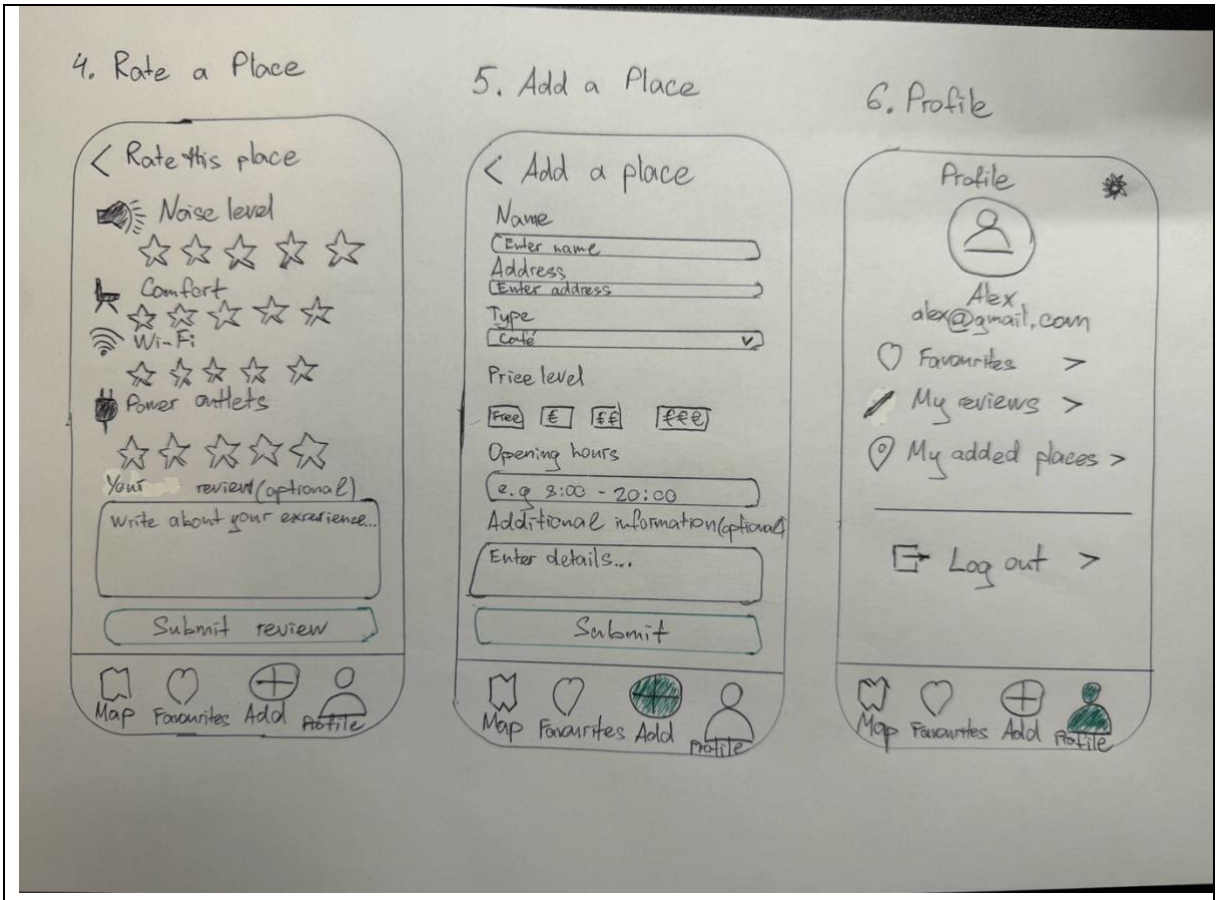
The profile screen allows users to manage their personal activity.

It includes:

- User profile
- Favorite locations
- Previously visited locations
- User's reviews and ratings
- Places added by the user

This screen provides access to the user's personal information and activity within the application.



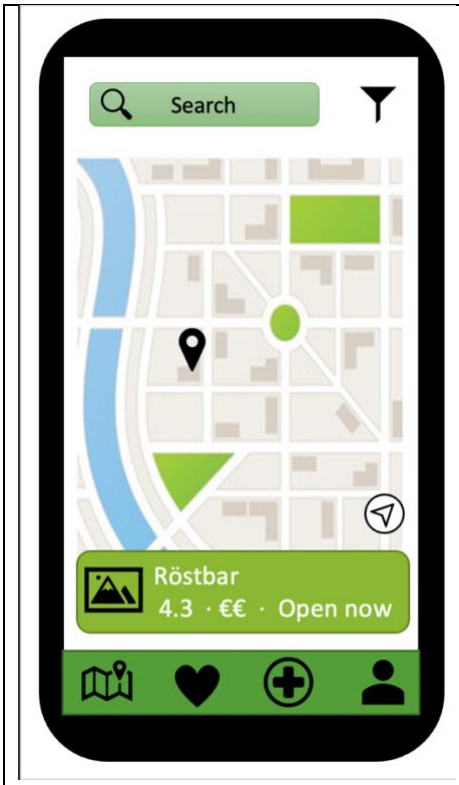


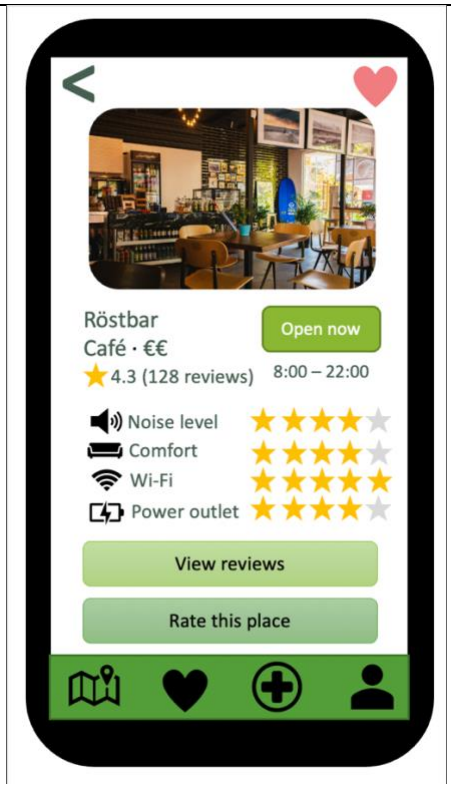
Create a digital Mock-Up

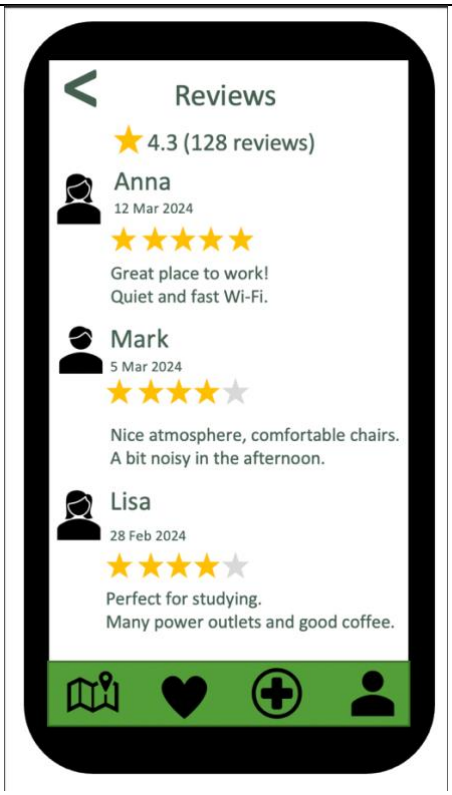
Create a **digital Mock-Up** based on your Paper Sketches:

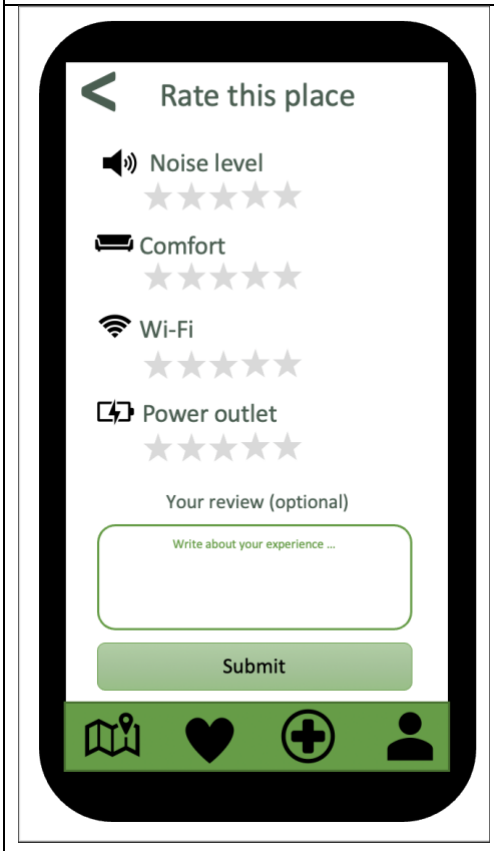
- Use **PowerPoint** for your digital Mock-Up.
- Create **at least 6 Screens** using PowerPoint.
- Define **Navigation** between Screens using **Active Links.**
- **Insert images** of your digital Mock-Up here.

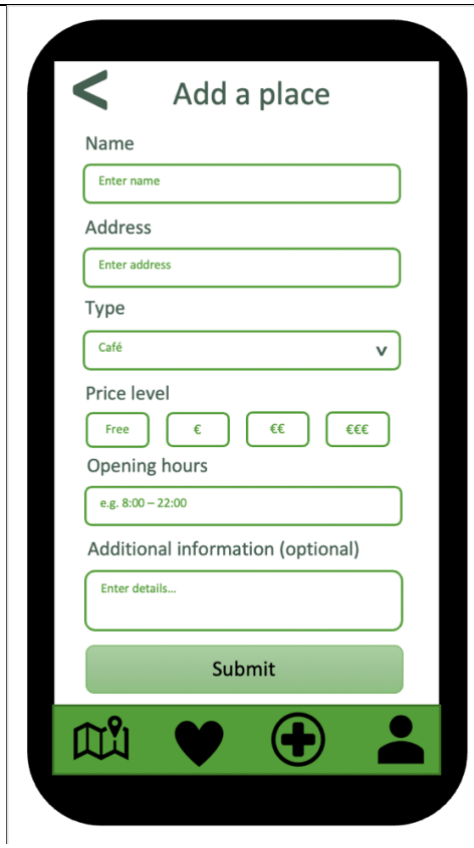
Alternatively, you can use another Tool for your digital Mock-Up, provided it supports the requirements.

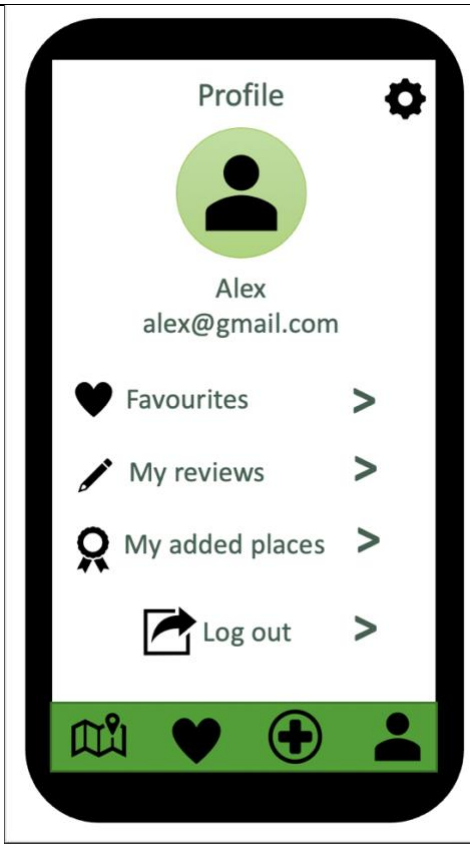

Search
Röstbar
4.3 · €€ · Open now


Röstbar
Café · €€
4.3 (128 reviews)
Open now
8:00 – 22:00
Noise level
Comfort
Wi-Fi
Power outlet
View reviews
Rate this place


Reviews
4.3 (128 reviews)
Anna
12 Mar 2024
Great place to work!
Quiet and fast Wi-Fi.
Mark
5 Mar 2024
Nice atmosphere, comfortable chairs.
A bit noisy in the afternoon.
Lisa
28 Feb 2024
Perfect for studying.
Many power outlets and good coffee.


Rate this place
Noise level
Comfort
Wi-Fi
Power outlet
Your review (optional)
Write about your experience ...
Submit


Add a place
Name
Enter name
Address
Enter address
Type
Café
Price level
Free
€
€€
€€€
Opening hours
e.g. 8:00 – 22:00
Additional information (optional)
Enter details...
Submit


Profile
Alex
alex@gmail.com
Favourites
My reviews
My added places
Log out

Configure your Development Environment

Install a Development Environment and explore Examples:

Attach Screenshots of your installation process and **describe your experience**.

- **Apple iOS**: Xcode ☞ Swift (or Objective-C)

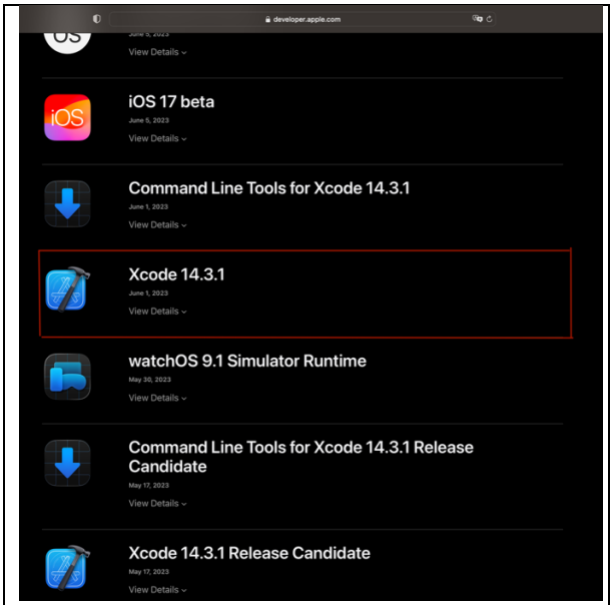
  https://developer.apple.com/tutorials/app-dev-training

- **Android**: Android Studio ☞ Java or Kotlin

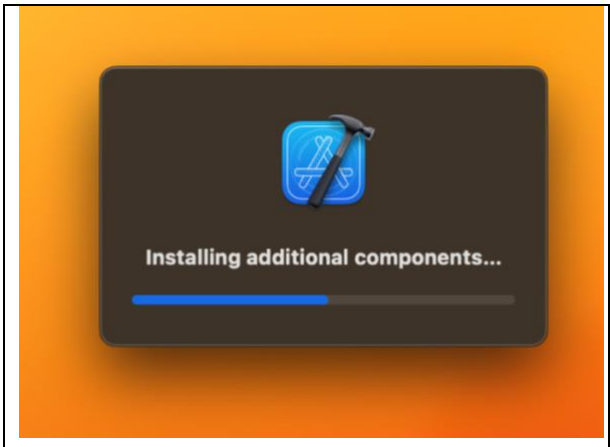
  https://developer.android.com

☞ No other Development Environment such as MIT APP Inventor is permitted!

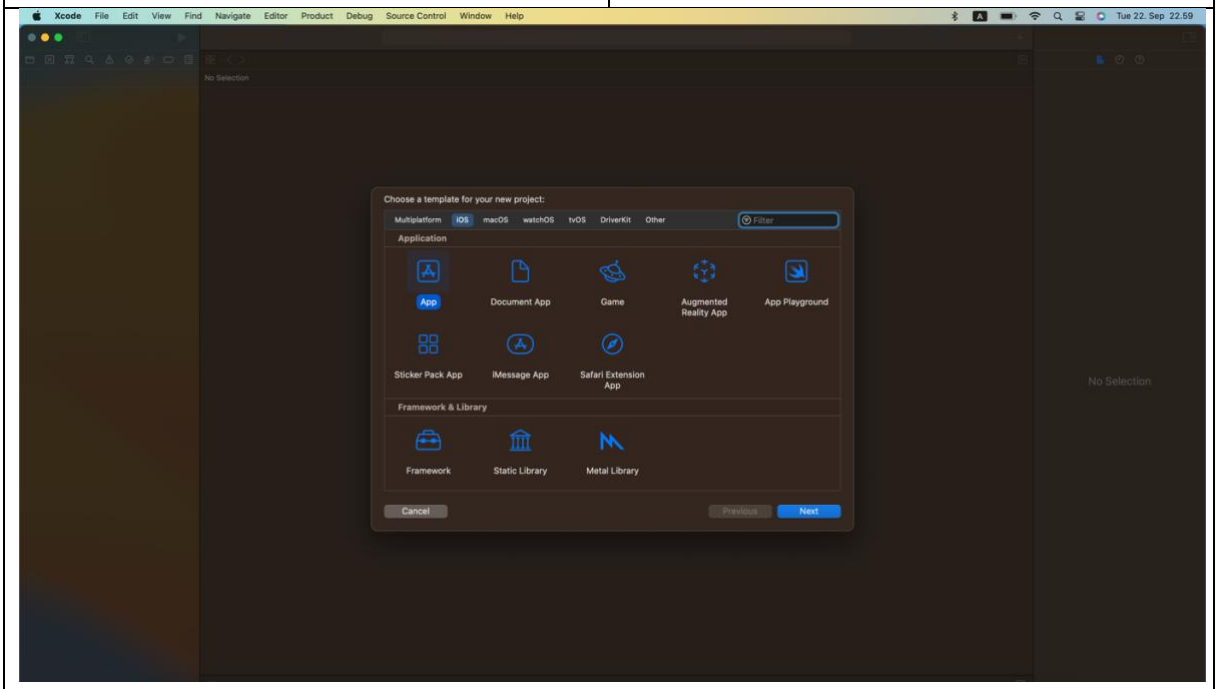
First, I checked which version of Xcode was compatible with the macOS version installed on my Mac. I then downloaded and installed the appropriate version of Xcode and accepted the required Apple license agreement. The installation process was simple and straightforward. After completing the installation, I was able to launch Xcode, create a new project, and explore its development environment.











Develop the Requirements and Steps for App Distribution

Describe in detail what is required to **distribute apps in a store**:

- **Apple App Store** or
- **Google Play Store**

Requirements and Steps for App Distribution

Apple App Store:

- Enrol in the Apple Developer Program.
- Prepare the app, icon, screenshots, description and privacy information.
- Build and test the app in Xcode.
- Upload it to App Store Connect.
- Submit it for Apple's review and publish it after approval.

Implement your Design Proposal

Your **implementation must run on Apple iOS or Android** (required)!

- You must implement **at least 3 Screens** (Displays).
- **Navigation between the Displays** must work.
- Your implementation should serve as a **working Prototype** that can be further developed into a First Release.
- Your implementation should **run on different Devices** (Smartphones, Tablets) depending on the chosen platform (simulator).
- **Attach Screenshots** of your working **Prototype** as well as Screenshots of associated **Programming Code** and **describe how it works**.

☞ Describe your experience!

Implementation of the WorkSpot Prototype

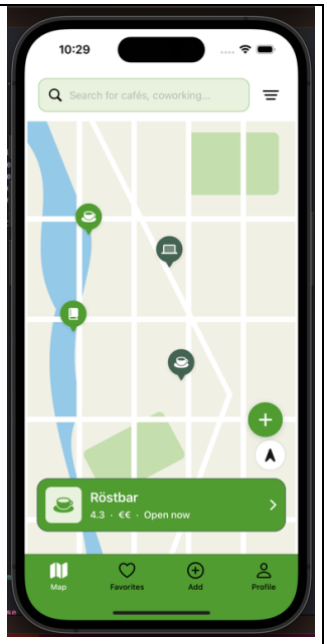
1. Map and Place Selection

```
            }
            .accessibilityLabel("Filters")
        }
        .padding(.horizontal, 20)
        .padding(.top, 12)
        .padding(.bottom, 10)

        ZStack(alignment: .bottom) {
            MapArtwork()
                .ignoresSafeArea(edges: .bottom)
                .contentShape(Rectangle())
                .onTapGesture {
                    showMapPlaceCard = false
                }

            // Only the Röstbar marker is interactive: it reveals the preview card.
            Button {
                showMapPlaceCard = true
            } label: {
                MapPin(symbol: "cup.and.saucer.fill", color: WorkSpotStyle.green)
            }
            .buttonStyle(.plain)
            .accessibilityLabel("Show Röstbar preview")
            .position(x: 92, y: 145)

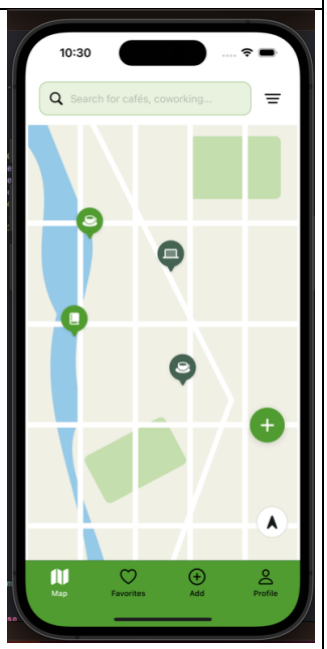
            // Decorative map markers; they do not open any place.
            MapPin(symbol: "laptopcomputer", color: WorkSpotStyle.darkGreen)
                .allowsHitTesting(false)
                .position(x: 208, y: 192)
```




```
            if showMapPlaceCard {
                Button { openPlace() } label: {
                    HStack(spacing: 12) {
                        RoundedRectangle(cornerRadius: 8)
                        .fill(WorkSpotStyle.paleGreen)
                        .frame(width: 52, height: 52)
                        .overlay {
                            Image(systemName: "cup.and.saucer.fill")
                                .font(.system(size: 23))
                                .foregroundStyle(WorkSpotStyle.green)
                        }

                    VStack(alignment: .leading, spacing: 5) {
                        Text("Röstbar")
                            .font(.system(size: 19, weight: .semibold))
                        Text("4.3  ·  €€  ·  Open now")
                            .font(.system(size: 14, weight: .medium))
                    }
                    .foregroundStyle(.white)

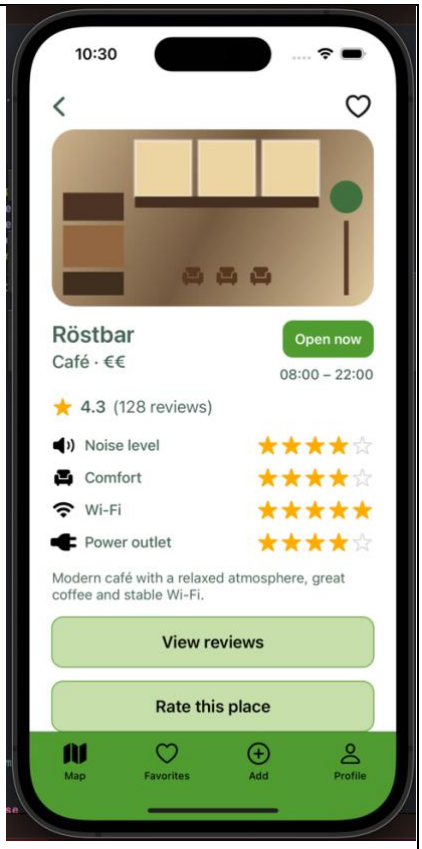
                    Spacer(minLength: 0)
                    Image(systemName: "chevron.right")
                        .font(.system(size: 17, weight: .semibold))
                        .foregroundStyle(.white)
                }
                    .padding(12)
                    .background(WorkSpotStyle.green)
                    .clipShape(RoundedRectangle(cornerRadius: 16))
                    .overlay(RoundedRectangle(cornerRadius: 16).stroke(WorkSpotStyle.darkGreen, lineWidth: 1))
                }
            }
        }
```



The map uses state variables to control the visibility of the Röstbar preview card. Tapping the Röstbar marker sets showMapPlaceCard to true, while tapping the map sets it to false. The preview card contains a button that opens the full place details.

2. Place Details

```
280
281         ScrollView(showsIndicators: false) {
282             VStack(alignment: .leading, spacing: 15) {
283                 CafePhotoPlaceholder()
284                     .frame(height: 190)
285                     .clipShape(RoundedRectangle(cornerRadius: 25))
286
287                 HStack(alignment: .top, spacing: 12) {
288                     VStack(alignment: .leading, spacing: 3) {
289                         Text(place.name)
290                             .font(.system(size: 25, weight: .semibold))
291                         Text("\(place.type) · \(place.price)")
292                             .font(.system(size: 19, weight: .medium))
293                     }
294                     .foregroundStyle(WorkSpotStyle.darkGreen)
295
296                     Spacer()
297
298                     VStack(alignment: .trailing, spacing: 9) {
299                         Text("Open now")
300                             .font(.system(size: 15, weight: .semibold))
301                             .foregroundStyle(.white)
302                             .padding(.horizontal, 13)
303                             .padding(.vertical, 11)
304                             .background(WorkSpotStyle.green)
305                             .clipShape(RoundedRectangle(cornerRadius: 11))
306                         Text(place.openingHours)
307                             .font(.system(size: 15, weight: .medium))
308                             .foregroundStyle(WorkSpotStyle.darkGreen)
309                     }
310                 }
311
312                 HStack(spacing: 7) {
313                     Image(systemName: "star.fill").foregroundStyle(WorkSpotStyle.gold)
314                     Text(place.rating).fontWeight(.semibold)
315                     Text("(\(place.reviewCount) reviews)")
316                 }
317                 .font(.system(size: 18))
318                 .foregroundStyle(WorkSpotStyle.darkGreen)
319
320                 VStack(spacing: 12) {
321                     RatingRow(icon: "speaker.wave.2.fill", title: "Noise level", rating: place.noise)
322                     RatingRow(icon: "chair.lounge.fill", title: "Comfort", rating: place.comfort)
323                     RatingRow(icon: "wifi", title: "Wi-Fi", rating: place.wifi)
324                     RatingRow(icon: "powerplug.fill", title: "Power outlet", rating: place.power)
325                 }
326                 .padding(.top, 4)
327
328                 Text(place.description)
329                     .font(.system(size: 15))
330                     .foregroundStyle(WorkSpotStyle.darkGreen)
331                     .fixedSize(horizontal: false, vertical: true)
332                     .padding(.top, 3)
```



The details screen displays information stored in the selected Place object, including its name, price level, opening hours, and ratings. Reusable RatingRow components display the individual ratings for noise level, comfort, Wi-Fi, and power outlets.

3. Reviews

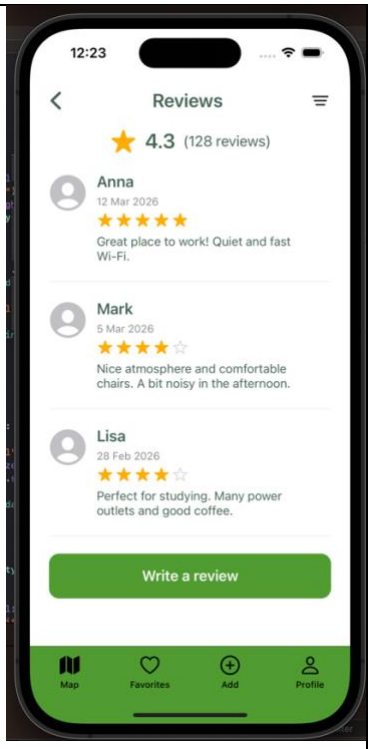
```swift
// MARK: Reviews

private var reviewsScreen: some View {
    VStack(spacing: 0) {
        HStack {
            Button { showReviews = false } label: {
                Image(systemName: "chevron.left")
                    .font(.system(size: 25, weight: .semibold))
                    .foregroundStyle(WorkSpotStyle.darkGreen)
            }
            Spacer()
            Text("Reviews")
                .font(.system(size: 23, weight: .semibold))
                .foregroundStyle(WorkSpotStyle.darkGreen)
            Spacer()
            Image(systemName: "line.3.horizontal.decrease")
                .font(.system(size: 20, weight: .medium))
                .foregroundStyle(WorkSpotStyle.ink)
        }
        .padding(.horizontal, 24)
        .padding(.top, 15)
        .padding(.bottom, 20)

        ScrollView(showsIndicators: false) {
            VStack(alignment: .leading, spacing: 22) {
                HStack(spacing: 9) {
                    Image(systemName: "star.fill").foregroundStyle(WorkSpotStyle.gold).font(.system(size: 27))
                    Text("4.3").font(.system(size: 25, weight: .semibold))
                    Text("(128 reviews)").font(.system(size: 18))
                }
                .foregroundStyle(WorkSpotStyle.darkGreen)
                .frame(maxWidth: .infinity)

                ForEach(reviews) { review in
                    ReviewRow(review: review)
                    Divider().overlay(WorkSpotStyle.gray.opacity(0.5))
                }

                Button { openRatePlace() } label: {
                    FormActionButton(title: "Write a review", filled: true)
                }
                .padding(.bottom, 100)
            }
            .padding(.horizontal, 24)
        }
    }
}
```



The Reviews screen displays the overall rating and a list of user reviews. Each review contains the user's name, date, star rating, and written comment. The screen uses SwiftUI views to arrange this information in a readable format.

4. Rate a place

```swift
// MARK: Rate a Place

private var ratePlaceScreen: some View {
    VStack(spacing: 0) {
        PageHeader(title: "Rate this place") {
            showRatingScreen = false
        }

        ScrollView(showsIndicators: false) {
            VStack(spacing: 20) {
                StarScaleRow(icon: "speaker.wave.2.fill", title: "Noise level", rating: $noiseRating)
                StarScaleRow(icon: "chair.lounge.fill", title: "Comfort", rating: $comfortRating)
                StarScaleRow(icon: "wifi", title: "Wi-Fi", rating: $wifiRating)
                StarScaleRow(icon: "battery.100percent", title: "Power outlet", rating: $powerRating)

                Text("Your review (optional)")
                    .font(.system(size: 18, weight: .medium))
                    .foregroundStyle(WorkSpotStyle.darkGreen)
                    .padding(.top, 8)

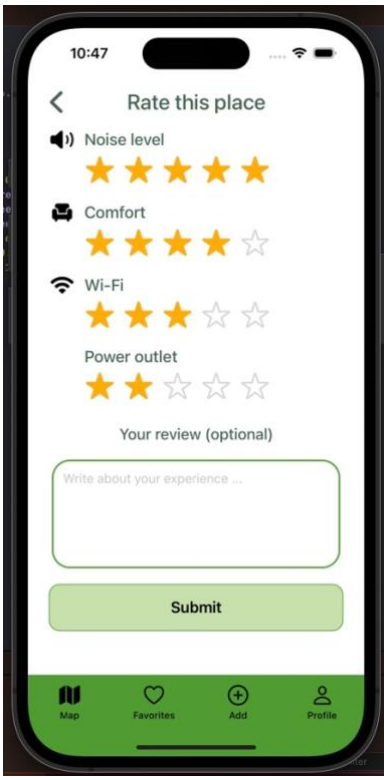
                TextField("Write about your experience ...", text: $reviewText, axis: .vertical)
                    .font(.system(size: 15))
                    .foregroundStyle(WorkSpotStyle.green)
                    .multilineTextAlignment(.center)
                    .lineLimit(4...6)
                    .padding(12)
                    .frame(minHeight: 125, alignment: .top)
                    .overlay(RoundedRectangle(cornerRadius: 20).stroke(WorkSpotStyle.green, lineWidth: 2))
                    .padding(.horizontal, 4)

                Button {
                    showRatingScreen = false
                    showReviews = true
                    showDetails = false
                } label: {
                    FormActionButton(title: "Submit", filled: false)
                }
                .padding(.bottom, 105)
            }
            .padding(.horizontal, 25)
        }
    }
}
```

```swift
private struct StarScaleRow: View {
    let icon: String
    let title: String
    @Binding var rating: Int

    var body: some View {
        VStack(alignment: .leading, spacing: 5) {
            HStack(spacing: 11) {
                Image(systemName: icon)
                    .font(.system(size: 24, weight: .medium))
                    .foregroundStyle(.black)
                    .frame(width: 30)
                Text(title)
                    .font(.system(size: 19, weight: .medium))
                    .foregroundStyle(WorkSpotStyle.darkGreen)
            }

            HStack(spacing: 7) {
                ForEach(1...5, id: \.self) { star in
                    Button { rating = star } label: {
                        Image(systemName: star <= rating ? "star.fill" : "star")
                            .font(.system(size: 30))
                            .foregroundStyle(star <= rating ? WorkSpotStyle.gold : WorkSpotStyle.gray)
                    }
                    .buttonStyle(.plain)
                    .accessibilityLabel("\(star) out of 5 stars")
                }
            }
            .padding(.leading, 40)
        }
        .frame(maxWidth: .infinity, alignment: .leading)
    }
}
```



The StarScaleRow component uses a loop to display five interactive stars for each rating category. The selected rating is updated through a binding variable, while @State manages the rating values and written review in the interface.

5. Add a place Form

```swift
// MARK: Add a Place

private var addPlaceScreen: some View {
    VStack(spacing: 0) {
        PageHeader(title: "Add a place") {
            showAddPlaceScreen = false
        }

        ScrollView(showsIndicators: false) {
            VStack(alignment: .leading, spacing: 12) {
                VStack(alignment: .leading, spacing: 11) {
                    AddFieldLabel(title: "Name")
                    TextField("Enter name", text: $newPlaceName).workSpotInput()

                    AddFieldLabel(title: "Address")
                    TextField("Enter address", text: $newPlaceAddress).workSpotInput()

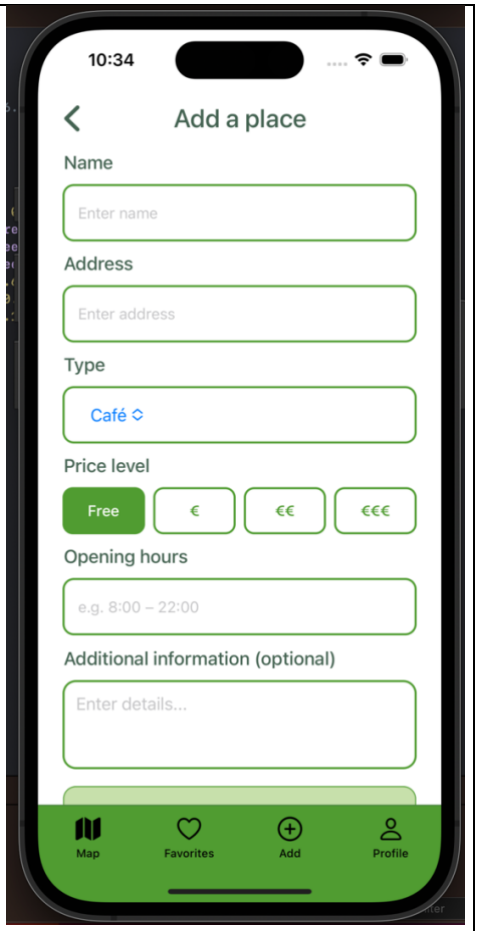
                    AddFieldLabel(title: "Type")
                    Picker("Type", selection: $newPlaceType) {
                        Text("Café").tag("Café")
                        Text("Coworking").tag("Coworking")
                        Text("Library").tag("Library")
                    }
                    .pickerStyle(.menu)
                    .frame(maxWidth: .infinity, alignment: .leading)
                    .frame(height: 54)
                    .padding(.horizontal, 14)
                    .overlay(RoundedRectangle(cornerRadius: 10).stroke(WorkSpotStyle.green, lineWidth: 2))

                    AddFieldLabel(title: "Price level")
                    HStack(spacing: 10) {
                        ForEach(["Free", "€", "€€", "€€€"], id: \.self) { level in
                            Button { newPlacePrice = level } label: {
                                Text(level)
                                    .font(.system(size: 15, weight: .medium))
                                    .foregroundStyle(newPlacePrice == level ? .white : WorkSpotStyle.green)
                                    .frame(maxWidth: .infinity)
                                    .frame(height: 44)
                                    .background(newPlacePrice == level ? WorkSpotStyle.green : .white)
                                    .clipShape(RoundedRectangle(cornerRadius: 9))
                                    .overlay(RoundedRectangle(cornerRadius: 9).stroke(WorkSpotStyle.green, lineWidth: 2))
                            }
                            .buttonStyle(.plain)
                        }
                    }

                    AddFieldLabel(title: "Opening hours")
                    TextField("e.g. 8:00 – 22:00", text: $newPlaceHours).workSpotInput()
                }

                VStack(alignment: .leading, spacing: 11) {
                    AddFieldLabel(title: "Additional information (optional)")
                    TextField("Enter details...", text: $newPlaceInfo, axis: .vertical)
                        .lineLimit(3...4)
                        .padding(12)
                        .frame(minHeight: 82, alignment: .topLeading)
                        .overlay(RoundedRectangle(cornerRadius: 12).stroke(WorkSpotStyle.green, lineWidth: 2))

                    Button {
                        showAddPlaceScreen = false
                        selectedTab = .map
                        clearAddPlaceForm()
                    } label: {
                        FormActionButton(title: "Submit", filled: false)
                    }
                    .padding(.top, 7)
                    .padding(.bottom, 105)
                }
            }
            .font(.system(size: 17))
            .foregroundStyle(WorkSpotStyle.darkGreen)
            .padding(.horizontal, 25)
```



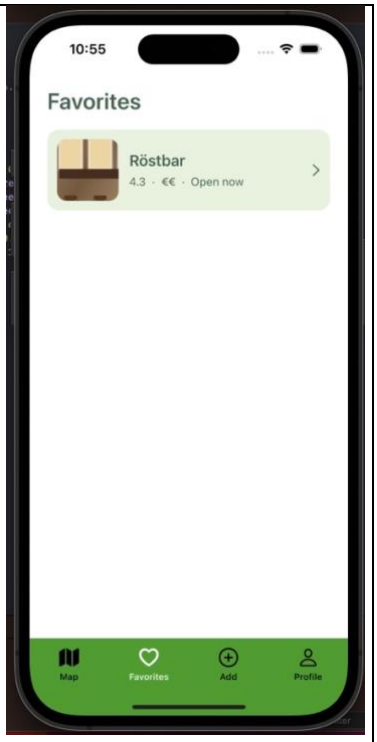
The form uses SwiftUI input fields bound to state variables to collect the place name, address, type, price level, opening hours, and additional information. The price buttons update the selected price level. The form demonstrates data entry, but submitting it does not permanently save a new place to a database.

6. Favorites / Profile

```
    // MARK: Favorites

    private var favoritesScreen: some View {
        VStack(alignment: .leading, spacing: 18) {
            Text("Favorites")
                .font(.system(size: 28, weight: .semibold))
                .foregroundStyle(WorkSpotStyle.darkGreen)
                .padding(.top, 18)

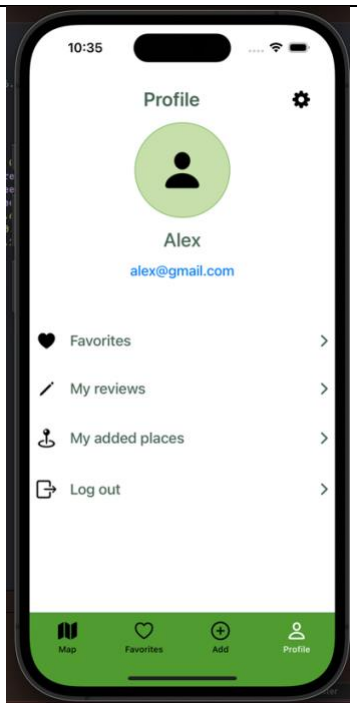
            if isFavorite {
                Button { openPlace() } label: {
                    HStack(spacing: 13) {
                        CafePhotoPlaceholder()
                            .frame(width: 76, height: 76)
                            .clipShape(RoundedRectangle(cornerRadius: 12))
                        VStack(alignment: .leading, spacing: 6) {
                            Text("Röstbar").font(.system(size: 19, weight: .semibold))
                            Text("4.3 · €€ · Open now").font(.system(size: 14))
                        }
                        .foregroundStyle(WorkSpotStyle.darkGreen)
                        Spacer()
                        Image(systemName: "chevron.right").foregroundStyle(WorkSpotStyle.darkGreen)
                    }
                    .padding(12)
                    .background(WorkSpotStyle.paleGreen)
                    .clipShape(RoundedRectangle(cornerRadius: 15))
                }
                .buttonStyle(.plain)
            } else {
                VStack(spacing: 12) {
                    Image(systemName: "heart").font(.system(size: 40)).foregroundStyle(WorkSpotStyle.green)
                    Text("No favorites yet").font(.system(size: 18, weight: .semibold)).foregroundStyle(WorkSpotStyle.darkGreen)
                    Text("Tap the heart on a place to save it here.")
                        .font(.system(size: 15))
                        .foregroundStyle(.secondary)
                        .multilineTextAlignment(.center)
                }
                .frame(maxWidth: .infinity)
                .padding(.top, 80)
            }
            Spacer()
        }
        .padding(.horizontal, 22)
        .padding(.bottom, 90)
    }
```




```
private struct ProfileRow: View {
    let icon: String
    let title: String
    var isEnabled: Bool = true
    let action: () -> Void

    private var rowContent: some View {
        HStack(spacing: 16) {
            Image(systemName: icon)
                .font(.system(size: 22, weight: .medium))
                .frame(width: 28)
                .foregroundStyle(.black)
            Text(title)
                .font(.system(size: 19, weight: .medium))
                .foregroundStyle(WorkSpotStyle.darkGreen)
            Spacer()
            Image(systemName: "chevron.right")
                .font(.system(size: 17, weight: .semibold))
                .foregroundStyle(WorkSpotStyle.darkGreen)
        }
        .padding(.vertical, 19)
        .contentShape(Rectangle())
    }

    var body: some View {
        Group {
            if isEnabled {
                Button(action: action) {
                    rowContent
                }
                .buttonStyle(.plain)
            } else {
                // Keep the same colors and layout, but render as a non-interactive row.
                rowContent
            }
        }
    }
}
```



The Favorites screen displays places saved by the user. The Profile screen shows the user's account information and available profile options. The bottom navigation allows users to switch between the main sections of the app. The reusable ProfileRow component displays profile options, while the isEnabled property determines whether a row responds to taps. My Reviews and My Added Places remain visible but inactive.

Print as a PDF Document

Print this Word Document **as a PDF Document** and send it to

- office@hollomey.com

☞ This is necessary to obtain a positive assessment!